

Programming Lab

Lezione 5

Accesso ed Ereditarietà

Laura Nenzi

Accesso

Pubblico e privato

- Alcuni degli **attributi** e dei **metodi** potrebbero essere specifici dell'implementazione e vorremmo non renderli visibili all'esterno:
- E.g., se usiamo una lista per implementare una coda vorremmo evitare che dall'esterno si acceda direttamente alla lista
- Alcuni linguaggi di programmazione forzano il controllo degli accessi con i modificatori **public**, **protected** e **private**
 - **Public**: accessibile a tutti
 - **Protected**: accessibili alla classe stessa e alle sottoclassi
 - **Private**: accessibili solo dalla classe stessa

Accesso

Pubblico e privato

- L'utilizzo di metodi e attributi privati ci permette di cambiare l'implementazione senza dover cambiare chi usa la classe...
- ...perché non può accedere ai dettagli dell'implementazione
- Però Python non supporta i modificatori “public”, “protected” e “private”
- Si utilizzano una serie di convenzioni per segnalare che alcuni attributi e metodi non sono pubblici

Accesso

Pubblico e privato

- *Metodi/attributi pubblici*: Nessun cambiamento
- *Metodi/attributi protetti*

Il nome del metodo/attributo inizia con “_”: `def _some_stuff(self):`
Possiamo comunque accedere senza problemi

- *Metodi/attributi privati*: Il nome del metodo/attributo inizia con “__” (doppio underscore): `def __this_is_private(self):`

In questo caso viene effettuato del name mangling:

il nome con cui il metodo/attributo viene visto all'esterno è
`_nomeclasse__nomemetodo`

Esempio di attributo protetto

```
class Queue:

    def __init__(self):
        self._data = []

    def enqueue(self, x):
        self._data.append(x)

    def dequeue(self):
        return self._data.pop(0)
```

```
q = Queue()
q.enqueue(5)
q.dequeue()
```

Tecnicamente si potrebbe fare `print(q._data.pop(0))` ma non è buona pratica e può portare ad errori ad esempio

Esempio di attributo protetto

- Ora voglio cambiare l'implementazione. Ora la coda non usa più una lista ma una deque

```
from collections import deque
```

```
class Queue:
```

```
    def __init__(self):  
        self._data = deque()
```

```
    def enqueue(self, x):  
        self._data.append(x)
```

```
    def dequeue(self):  
        return self._data.popleft()
```

```
q = Queue()  
q.enqueue(5)  
q.dequeue()
```

Funziona

```
q._data.pop(0)
```

Mi darà errore

Esempio di attributo privato

```
class Banca:  
  
    def __init__(self, saldo):  
        self.__saldo = saldo  
  
    def deposita(self, x):  
        self.__saldo += x  
  
    def saldo(self):  
        return self.__saldo
```

```
b = Banca(100)  
print(b.saldo())
```

Se provate a fare `print(b.__saldo)` vi darà errore.

Questo succede perché Python applica il **name mangling**.

Il nome reale diventa: `print(b._Banca__saldo)` ma farlo significa forzare il sistema

Accesso

Pubblico e privato

- Se deve essere accessibile all'esterno: pubblico
- Se è un dettaglio implementativo: protetto o privato (dipende come vogliamo che sia l'accesso da parte di classi che estendono quella attuale — uno dei prossimi argomenti)
- Se rendete qualcosa pubblico verrà utilizzato e sarà difficile cambiarlo dopo senza “rompere” del codice che usa la classe
- Se però rendete troppe funzionalità private potreste rendere la classe difficile da utilizzare (e.g., una coda senza “isempty”)

Metodi di una classe

Chiamare metodi della classe

- Un metodo di classe è un metodo che è della classe e non dell'oggetto:

- Metodi normali:

```
q = Queue()  
q.enqueue(3)
```

- Metodo di classe:

```
Queue.do_stuff()
```

- Perché dovrebbero servirci?

Metodi di una classe

Chiamare metodi della classe

- I metodi di classe possono essere chiamati prima che sia creato un oggetto...
- ...quindi possiamo usarli per fornire metodi alternativi di costruzione di oggetti:
 - Costruttore “standard”
 - Costruttore copiando da altro oggetto o struttura dati differente
- Evitare di creare una funzione globale per lavorare solo su oggetti di un tipo specifico

Metodi di una classe

Chiamare metodi della classe

- Un metodo di classe è creato precedendo la definizione con **@classmethod** (il primo argomento per convenzione si chiamerà “cls” e non “self”)

```
@classmethod  
def from_list(cls, lst):  
    q = cls()  
    q.__data = lst  
    return q
```

Qui verrà passata
la classe, non l'oggetto

Invochiamo il
costruttore “standard”

Metodi di una classe

Chiamare metodi della classe

- Nella maggior parte del codice Python i metodi di classe si usano molto meno dei metodi normali. Ma diventano molto utili quando si vogliono creare costruttori alternativi. E.g.:

```
class Person:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, s):
        name, age = s.split(',')
        return cls(name, int(age))
```

Dunder (double underscore) Methods

Metodi speciali

- Alcuni metodi hanno un significato speciale:
 - `__init__` per inizializzare un oggetto
 - `__str__` per avere una rappresentazione in stringa dell'oggetto (per funzioni come print)
- Esistono molti altri metodi di questo tipo che ci permettono, per esempio, di definire:
 - Il comportamento con le normali operazioni (somma, prodotto, divisione, etc.
 - Accesso con indici

Dunder Methods

Confronto tra oggetti

Metodo	Utilizzo
<code>__lt__</code>	<code>obj < ...</code>
<code>__le__</code>	<code>obj <= ...</code>
<code>__eq__</code>	<code>obj == ...</code>
<code>__ne__</code>	<code>obj != ...</code>
<code>__gt__</code>	<code>obj > ...</code>
<code>__ge__</code>	<code>obj >= ...</code>

Tutti questi metodi prendono due argomenti (self e un altro oggetto) e ritornano un valore Booleano

Dunder Methods

Confronto tra oggetti

```
class Student:

    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def __lt__(self, other):
        return self.grade < other.grade

    def __eq__(self, other):
        return self.grade == other.grade
```

```
s1 = Student("Anna", 28)
s2 = Student("Marco", 30)

print(s1 < s2) # True
print(s1 == s2) # False
```

Dunder Methods

Accesso con chiavi

Metodo	Utilizzo	Argomenti
<code>__getitem__</code>	Lettura di <code>obj[key]</code>	(self, key)
<code>__setitem__</code>	<code>obj[key] = ...</code>	(self, key, value)
<code>__delitem__</code>	<code>del obj[key]</code>	(self, key)

Questi metodi permettono di accedere a dei valori usando una chiave, quindi potremmo utilizzarli per implementare dizionari, matrici sparse, etc

Dunder Methods

Confronto tra oggetti

```
class MyDict:

    def __init__(self):
        self.data = {}

    def __getitem__(self, key):
        return self.data[key]

    def __setitem__(self, key, value):
        self.data[key] = value

    def __delitem__(self, key):
        del self.data[key]
```

```
d = MyDict()

d["a"] = 10      # __setitem__
print(d["a"])    # __getitem__
del d["a"]       # __delitem__
```

Dunder Methods

Operazioni matematiche

Metodo	Utilizzo	Argomenti
<code>__add__</code>	<code>obj + ...</code>	(self, other)
<code>__radd__</code>	<code>... + obj</code>	(self, other)
<code>__iadd__</code>	<code>obj += ...</code>	(self, other)
<code>__sub__</code>	<code>obj - ...</code>	(self, other)
<code>__mul__</code>	<code>obj * ...</code>	(self, other)

Se volessimo implementare operazioni di somma tra vettori ci basterebbe definire i metodi corrispondenti (ci sono anche metodi per divisione, modulo, elevamento a potenza, etc)

Dunder Methods

Confronto tra oggetti

```
class Vector:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, other):
        return Vector(self.x * other, self.y * other)
```

```
v1 = Vector(1,2)
v2 = Vector(3,4)
```

```
v3 = v1 + v2
print(v3.x, v3.y)
```

```
v4 = v1 * 3
print(v4.x, v4.y)
```

Dunder Methods

Altri metodi utili

Metodo	Utilizzo	Argomenti
<code>__len__</code>	<code>len(obj)</code>	<code>(self)</code>
<code>__contains__</code>	<code>... in obj</code>	<code>(self, other)</code>
<code>__call__</code>	<code>obj(args)</code>	<code>(self, ...)</code>

Di particolare interesse è il metodo `__call__` che permette di far comportare l'oggetto come se fosse una funzione, facendo cose come:

```
class LinearFunction:
```

```
    def __init__(self, a, b):  
        self.a = a  
        self.b = b
```

```
    def __call__(self, x):  
        return self.a * x + self.b
```

```
f = LinearFunction(2,3)  
  
print(f(5))
```

Parentesi sull'uso dei moduli

Scrivere moduli

- Qualunque file che contenga codice Python può essere importato come modulo.
- Supponiamo di avere un file di nome `wc.py` che contiene il codice che segue:

```
def contarighe(nomefile):  
    conta = 0  
    for riga in open(nomefile):  
        conta += 1  
    return conta  
print(contarighe('wc.py'))
```

- legge se stesso e stampa il numero delle righe nel file

Scrivere moduli

- Potete importare il file così:

```
>>> import wc  
7  
  
>>> wc.contarighe('wc.py')  
7
```

- I programmi che verranno importati come moduli usano spesso questo costrutto:

```
if __name__ == '__main__':  
    print(contarighe('wc.py'))
```

- Il codice così è eseguito solo se avvio lo script non se lo importo

dunder names

- Python imposta automaticamente la variabile `__name__` quando si esegue o si importa uno script
 - `__name__ = __main__` se si esegue lo script
 - `__name__ = nomescript` se si importa il file python nomescript.py

!!! Differenze con Java e C++

- In **Java e C++** esiste l'**overloading dei costruttori**: puoi avere più costruttori con lo stesso nome ma parametri diversi. E.g. in Java
- Qui il linguaggio sceglie **automaticamente il costruttore giusto** in base ai parametri.
- Python **non ha overloading dei costruttori**.
Posso usare costruttori alternativi con `@classmethod`
O considerare parametri opzionali ma attenzione

```
class Point {  
    int x, y;  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int x) {  
        this.x = x;  
        this.y = 0;  
    }  
    Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

Esercizio

- Scrivete una definizione di una classe di nome Canguro con i metodi seguenti:
 1. Un metodo `__init__` che inizializza un attributo `lista_contenuto_tasca` con paramatro di default lista vuota.
 2. Un metodo di nome `intasca` che prende un oggetto di qualsiasi tipo e lo inserisce in `contenuto_tasca`.
 3. Un metodo `__str__` che restituisce una stringa di rappresentazione dell'oggetto Canguro e dei contenuti della tasca.
- Provate il codice creando due oggetti Canguro, assegnandoli a variabili di nome `can` e `guro`, e aggiungendo elementi a `can`. Cosa succede a `guro`?

!!! Attenzione ai parametri opzionali

- Vuoi permettere all'utente di passare una lista oppure, se non la passa, crearla tu, come puoi fare?

NO!

```
class A:  
    def __init__(self, data=[]):  
        self.data = data
```

SI :)

```
class A:  
    def __init__(self, data=None):  
        if data is None:  
            self.data = []  
        else:  
            self.data = data
```

- Mai usare oggetti mutabili (list, dict, set) come valori di default nei parametri

Ereditarietà

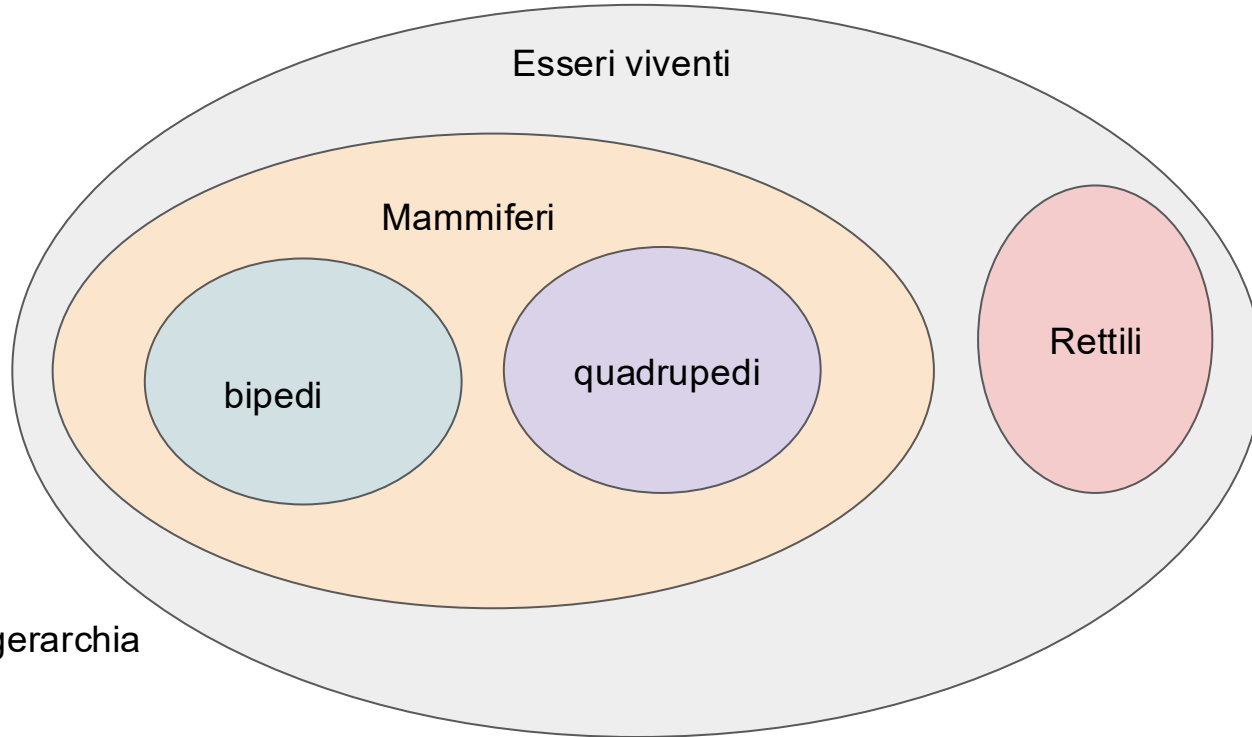
Estendere oggetti: l'ereditarietà

- Spesso una serie di classi modellano concetti simili...
- ...e quindi il contenuto dei metodi risulta praticamente duplicato
- Questo è dovuto al fatto che i metodi sono legati a una classe e quindi non possiamo dire direttamente “usa lo stesso metodo per entrambe queste classi”
- Potrebbe essere utile invece avere una **classe più generica** che contiene il codice comune...
- ...e classi più specializzate che ereditano le parti comuni

Estendere oggetti: l'ereditarietà

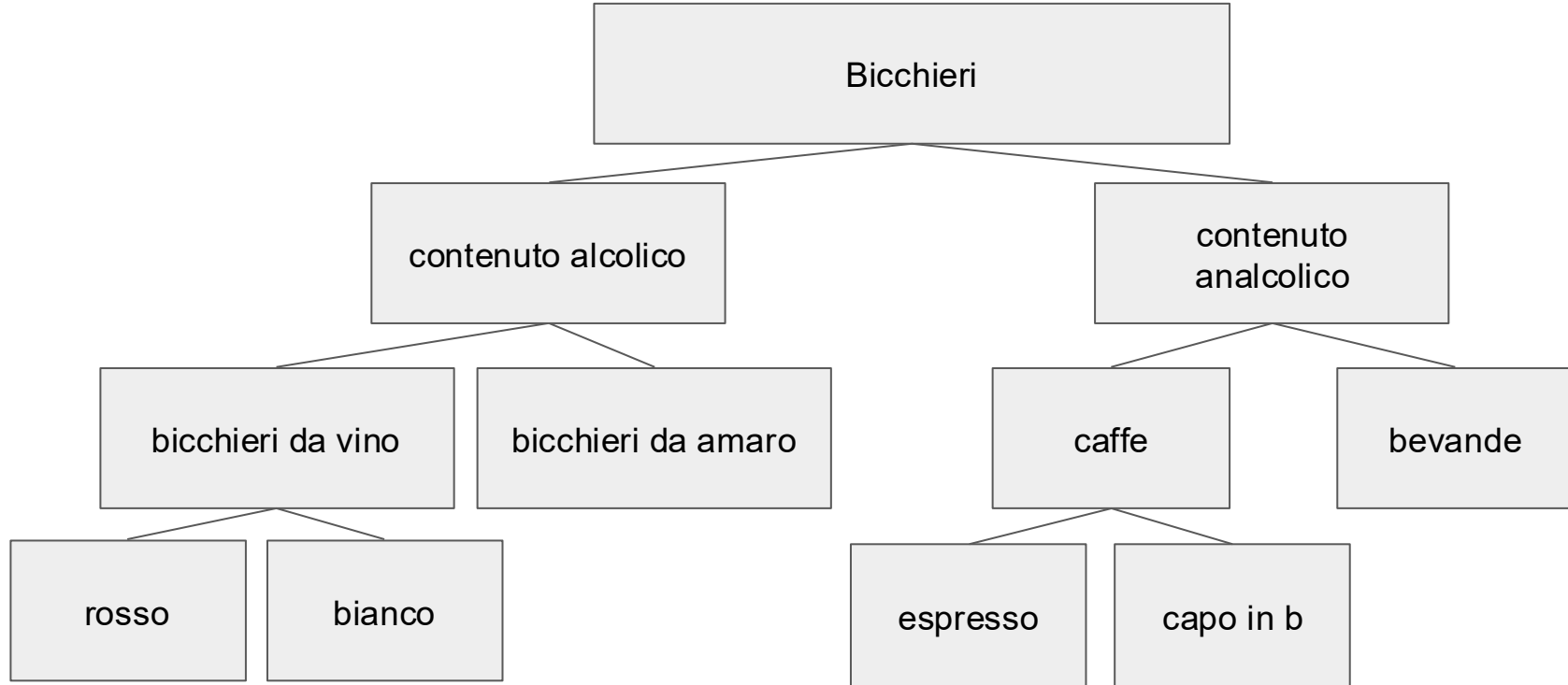
- L'**ereditarietà** è la capacità di definire una nuova classe come versione modificata di una classe già esistente
- Permette, a partire da una classe genitore, di creare classi figlie, cioè di definire nuove classi come **versione modificata di una classe già esistente**.
- Possiamo interpretare le gerarchie tra classi in termini di insiemi e relazioni di inclusione

Estendere oggetti : l'ereditarietà

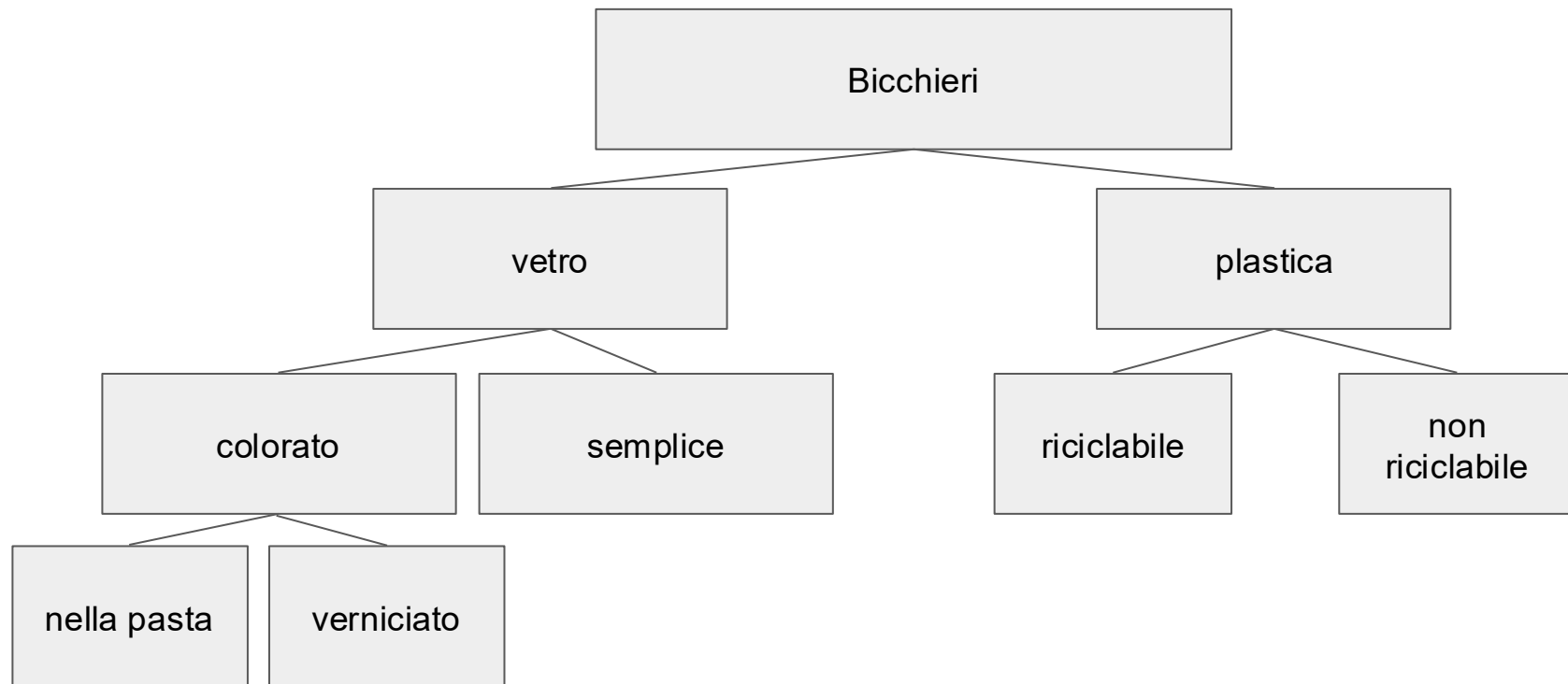


Esempio di gerarchia

Ereditarietà



Ereditarietà



Ereditarietà

- L'operazione di estensione di una classe prende anche il nome di **specializzazione**, nel senso che la si rende più “specificata” per una determinata entità che si vuole rappresentare
- Se la classe B eredita dalla classe A, si dice che B è una **sotto-classe** di A e che A è una **super-classe** di B. A seconda del linguaggio di programmazione, l'ereditarietà può essere **singola** (ogni classe può avere al più una super-classe) o **multipla** (ogni classe può avere più super-classi).
- La relazione è transitiva: in un programma potresti avere A super-classe di B, e B super-classe di C.

Ereditarietà

- Dire che una classe derivata eredita da una classe base è indicato nel seguente modo:

class Derived(Base):



Stiamo indicando che “Derived”
è una sottoclasse di “Base”

- Le classi figlie:
 - ereditano tutti gli attributi e metodi della classe genitore
 - possono sovrascrivere (**overriding**) i metodi ereditati per personalizzarli e possono aggiungerne di nuovi per estenderne le funzionalità.
- Le modifiche effettuate ad una classe figlia non impattano la classe genitore.

Ereditarietà

```
# Import the random module
import random

class Person():

    def __init__(self, name, surname):

        # Set name and surname
        self.name = name
        self.surname = surname

    def __str__(self):
        return 'Person "{} {}".format(self.name, self.surname)

    def say_hi(self):

        # Generate a random number between 0, 1 and 2.
        random_number = random.randint(0,2)

        # Choose a random greeting
        if random_number == 0:
            print('Hello, I am {} {}.'.format(self.name, self.surname))
        elif random_number == 1:
            print('Hi, I am {}'.format(self.name))
        elif random_number == 2:
            print('Yo bro! {} here!'.format(self.name))

person = Person('Mario', 'Rossi')
person.say_hi()
```

➤ python objects.py
Hello, I am Mario Rossi.

➤ █

➤ python objects.py
Hi, I am Mario!

➤ █

➤ python objects.py
Yo bro! Mario here!

➤ █

Ereditarietà

```
class Person():
    ...

class Student(Person):

    def __str__(self):
        return 'Student "{} {}".format(self.name, self.surname)

class Professor(Person):

    def __str__(self):
        return 'Prof. "{} {}".format(self.name, self.surname)

    def say_hi(self):
        print('Hello, I am professor {} {}.'.format(self.name, self.surname))
```

Ereditarietà

objects.py

```
class Person():
    ...

class Student(Person):

    def __str__(self):
        return 'Student "{} {}".format(self.name, self.surname)

class Professor(Person):

    def __str__(self):
        return 'Prof. "{} {}".format(self.name, self.surname)

    def say_hi(self):
        print('Hello, I am professor {} {}'.format(self.name, self.surname))

    def original_say_hi(self):
        super().say_hi()
```

Ereditarietà

objects.py

```
class Person():
    ...

class Student(Person):

    def __str__(self):
        return 'Student "{} {}".format(self.name, self.surname)

class Professor(Person):

    def __str__(self):
        return 'Prof. "{} {}".format(self.name, self.surname)

    def say_hi(self):
        print('Hello, I am professor {} {}'.format(self.name, self.surname))

    def original_say_hi(self):
        super().say_hi()
```

Con il “super” accedo alla funzione dell'oggetto padre, anche se l'ho sovrascritta

Ereditarietà

Esempio

objects.py

```
class Person():
    ...

class Student(Person):

    def __str__(self):
        return 'Student "{} {}".format(self.name, self.surname)

class Professor(Person):

    def __str__(self):
        return 'Prof. "{} {}".format(self.name, self.surname)

    def say_hi(self):
        print('Hello, I am professor {} {}'.format(self.name, self.surname))

    def original_say_hi(self):
        super().say_hi()
```

```
print('-----')

person = Person('Mario', 'Rossi')

print(person)
person.say_hi()

print('-----')

prof = Professor('Pippo', 'Baudo')

print(prof)
prof.say_hi()
prof.original_say_hi()

print('-----')
```

python objects.py

```
-----
Person "Mario Rossi"
Yo bro! Mario here!
-----
Prof. "Pippo Baudo"
Hello, I am professor Pippo Baudo.
Yo bro! Pippo here!
-----
```


Ereditarietà

- Se la classe base ha un costruttore `__init__()`, la classe figlia può averne uno diverso **con solo parte degli attributi.**
- In quel caso posso specificare **l'attributo della classe base** con `super().__init__()`

Ereditarietà

```
class Persona:
    def __init__(self, ruolo, nome, cognome):
        self.ruolo = ruolo
        self.nome = nome
        self.cognome = cognome

    def saluta(self):
        print('Ciao sono', self.ruolo + ",", self.nome, self.cognome)
```

```
class Studente(Persona):

    # costruttore sotto-classe
    def __init__(self, nome, cognome, corso):
        super().__init__("Studente UNITS", nome, cognome)
        self.corso = corso

    # ridefinizione del metodo bonjour di persona
    def saluta(self):
        Persona.saluta(self) # uso esplicitamente metodo di Persona
        print("> Frequento il corso: ", self.corso)
```

Ereditarietà

```
class Docente(Persona):  
    def __init__(self, nome, cognome, corso):  
        super().__init__("Docente UNITS", nome, cognome)  
        self.corso = corso  
  
    def saluta(self):  
        Persona.saluta(self)  
        print("> Docente del corso: ", self.corso)
```

Esercizio 1

- Si vogliono modificare le classi `Studente` e `Docente` precedentemente definite in modo che esse possano:
 - memorizzare una lista di corsi frequentata da uno studente
 - memorizzare una lista di corsi insegnati da un Docente
- Per semplicità le liste in questione non sono vuote. Si vuole quindi definire `saluta` per stampare a schermo la lista completa dei corsi.
- In pratica, si vuole permettere di scrivere il seguente codice:
 - `corsi = ["Programmazione", "Laboratorio", "Analisi", "Geometria"]`
 - `obj_Irene = Studente("Irene", "Rossi", corsi)`
- cosicché `obj_Irene.saluta()` stampi tutti i corsi frequentati da `obj_Irene` (variabile `corsi`, passata al costruttore).

Esercizio 2

- Crea una sottoclasse auto di veicolo che ha in aggiunta l'attributo **numero_porte** e cambia il metodo `__str__` di conseguenza
- Crea una sottoclasse moto che ha in aggiunta l'attributo **tipo** (ad esempio, "Sportiva" o "Touring") e cambia il metodo `__str__` di conseguenza

Esercizio 3

- Si estendano ulteriormente le classi definite nell'esercizio 1 aggiungendo:
 - un algoritmo che calcoli se un docente insegna tutti i corsi frequentati da uno studente.
 - un algoritmo che verifichi che, per ogni studente iscritto ad un certo numero di corsi, esistano docenti che effettivamente insegnino quei corsi.
- Per risolvere il secondo punto si usi la soluzione del primo.

Esercizio 4

- Si crei la classe Poligono:
 - Il costruttore deve prendere solo il numero di lati
 - Deve fornire una descrizione del tipo: "Sono un poligono con X lati"
- Definire una sottoclasse Quadrilatero, modificare il costruttore in modo opportuno e sovrascrivere la descrizione per essere: "Sono un quadrilatero"
- Definire una sottoclasse Rettangolo di Quadrilatero, la cui inizializzazione necessita di base ed altezza, modificare la descrizione per includere questa informazione e definire i metodi perimetro ed area
- Definire la classe Triangolo, la cui inizializzazione necessita di 3 lati. Modificare la descrizione per includere la lunghezza dei lati. Fornire un metodo perimetro e uno is_equilatero (restituisce True se il triangolo è equilatero)